模考总评

T1, T2

配对序列

安全接线

T3, T4

字符串生成器

极差之和

第 1 讲：CSP-S 模考 1

学而思 CSP-S 复赛模考讲评课

倪理涵

学而思素养中心

2026 年 10 月 11 日

题目大意

- 给定 n 个元素的序列 $a_1, \ldots, a_n$，n 为偶数，$a_i \in [1, m]$。
- 若存在整数 T，使得对任意 $1 \leq i \leq \frac{n}{2}$ ，都有 $a_i + a_{n-i+1} = T$，则称序列配对。
- 每次操作可将某个 a_i 改成 $[1, m]$ 中任意整数。
- 求使序列配对的最少操作次数。

得分情况

- 参考难度：普及 +/提高-。

问题分析

- 注意到测试点 $1 \sim 8$：$n, m \leq 100$。

问题分析

- 注意到测试点 $1 \sim 8$：$n, m \leq 100$。
  - 枚举 T，求使 $a_i + a_{n-i+1} = T$ 的操作次数；

问题分析

- 注意到测试点 $1 \sim 8$：$n, m \leq 100$。
  - 枚举 T，求使 $a_i + a_{n-i+1} = T$ 的操作次数；
  - T 的枚举范围应为 $[2, 2m]$。

问题分析

- 注意到测试点 $1 \sim 8$：$n, m \leq 100$。
  - 枚举 T，求使 $a_i + a_{n-i+1} = T$ 的操作次数；
  - T 的枚举范围应为 $[2, 2m]$。
- 对一对元素 (x, y)，要使其和变为 T：
  - $T = x + y$ 时，无需改动，次数为 0；

问题分析

- 注意到测试点 $1 \sim 8$：$n, m \leq 100$。
  - 枚举 T，求使 $a_i + a_{n-i+1} = T$ 的操作次数；
  - T 的枚举范围应为 $[2, 2m]$。
- 对一对元素 (x, y)，要使其和变为 T：
  - $T = x + y$ 时，无需改动，次数为 0；
  - 如果可以只改一个数，则次数为 1；
  - 否则，两个数都要改，次数为 2：

```
1  for (int nx = 1; nx <= m; ++nx)
2    if (nx + y == T) res = 1;
3
4  for (int ny = 1; ny <= m; ++ny)
5    if (x + ny == T) res = 1;
```

问题分析

- 注意到测试点 $1 \sim 8$：$n, m \leq 100$。
  - 枚举 T，求使 $a_i + a_{n-i+1} = T$ 的操作次数；
  - T 的枚举范围应为 $[2, 2m]$。
- 对一对元素 (x, y)，要使其和变为 T：
  - $T = x + y$ 时，无需改动，次数为 0；
  - 如果可以只改一个数，则次数为 1；
  - 否则，两个数都要改，次数为 2：

```
1  for (int nx = 1; nx <= m; ++nx)
2    if (nx + y == T) res = 1;
3
4  for (int ny = 1; ny <= m; ++ny)
5    if (x + ny == T) res = 1;
```

- 时间复杂度 $\mathcal{O}(nm^2)$，期望得分 40 分。

问题分析

- 注意到测试点 $1 \sim 12$：$n, m \leq 5000$。

问题分析

- 注意到测试点 $1 \sim 12$：$n, m \leq 5000$。
  - 考虑优化枚举，减少一层循环。

问题分析

- 注意到测试点 $1 \sim 12$：$n, m \leq 5000$。
  - 考虑优化枚举，减少一层循环。
- 次数为 1 时，改变后的 $x + y$ 取值范围：
  - x 变 y 不变：$[1 + y, m + y]$；

问题分析

- 注意到测试点 $1 \sim 12$：$n, m \leq 5000$。
  - 考虑优化枚举，减少一层循环。
- 次数为 1 时，改变后的 $x + y$ 取值范围：
  - x 变 y 不变：$[1 + y, m + y]$；
  - x 不变 y 变：$[1 + x, m + x]$。

问题分析

- 注意到测试点 $1 \sim 12$：$n, m \leq 5000$。
  - 考虑优化枚举，减少一层循环。
- 次数为 1 时，改变后的 $x + y$ 取值范围：
  - x 变 y 不变：$[1 + y, m + y]$；
  - x 不变 y 变：$[1 + x, m + x]$。
- 记 $l = 1 + \min(x, y), r = m + \max(x, y)$。
- 只要 $l \leq T \leq r$，次数就为 1；

问题分析

- 注意到测试点 $1 \sim 12$：$n, m \leq 5000$。
  - 考虑优化枚举，减少一层循环。
- 次数为 1 时，改变后的 $x + y$ 取值范围：
  - x 变 y 不变：$[1 + y, m + y]$；
  - x 不变 y 变：$[1 + x, m + x]$。
- 记 $l = 1 + \min(x, y), r = m + \max(x, y)$。
- 只要 $l \leq T \leq r$，次数就为 1；
- 时间复杂度 $\mathcal{O}(nm)$，期望得分 60 分。

问题分析

- 注意到测试点 $13 \sim 14$：a_i 全部相等。

问题分析

- 注意到测试点 $13 \sim 14$：a_i 全部相等。
  - 利用特殊性质，可进一步争取得分。

问题分析

- 注意到测试点 $13 \sim 14$：a_i 全部相等。
  - 利用特殊性质，可进一步争取得分。
- 记 $a_i = v$，则每一对都是 (v, v)，和恒为 $2v$。
  - 直接令 $T = 2v$，所有配对都无需改动，答案即为 0。

问题分析

- 注意到测试点 $13 \sim 14$：a_i 全部相等。
  - 利用特殊性质，可进一步争取得分。
- 记 $a_i = v$，则每一对都是 (v, v)，和恒为 $2v$。
  - 直接令 $T = 2v$，所有配对都无需改动，答案即为 0。
- 先判断，再根据结果，决定运行暴力还是特判：

```
1  bool all = true;
2  for (int i = 2; i <= n; ++i)
3      if (a[i] != a[1]) all = false;
4
5  if (all) { cout << 0 << '\n'; return 0; }
6  ...
```

问题分析

- 注意到测试点 $13 \sim 14$：a_i 全部相等。
  - 利用特殊性质，可进一步争取得分。
- 记 $a_i = v$，则每一对都是 (v, v)，和恒为 $2v$。
  - 直接令 $T = 2v$，所有配对都无需改动，答案即为 0。
- 先判断，再根据结果，决定运行暴力还是特判：

```
1  bool all = true;
2  for (int i = 2; i <= n; ++i)
3      if (a[i] != a[1]) all = false;
4
5  if (all) { cout << 0 << '\n'; return 0; }
6  ...
```

- 结合已有部分分，期望得分 70 分。

问题分析

- 为继续优化，应避免对每个 T，重新枚举所有对。

问题分析

- 为继续优化，应避免对每个 T，重新枚举所有对。
- 贡献法：$T \in [2, 2m]$ 时，每一对 (x, y) 对操作次数的贡献可视作区间：
  - $T = x + y$：贡献为 0；
  - $T \in [l, r]$：贡献为 1；
  - 其他情况：贡献为 2。

问题分析

- 为继续优化，应避免对每个 T，重新枚举所有对。
- 贡献法：$T \in [2, 2m]$ 时，每一对 (x, y) 对操作次数的贡献可视作区间：
  - $T = x + y$：贡献为 0；
  - $T \in [l, r]$：贡献为 1；
  - 其他情况：贡献为 2。
- 可使用差分，快速维护区间加法操作。
  - 差分还原后，答案即为各 T 值的操作次数最小值：

```
1 d[2] += 2;  d[2 * m + 1] -= 2;
2 --d[l];     ++d[r + 1];
3 --d[x + y]; ++d[x + y + 1];
```

问题分析

- 为继续优化，应避免对每个 T，重新枚举所有对。
- 贡献法：$T \in [2, 2m]$ 时，每一对 (x, y) 对操作次数的贡献可视作区间：
  - $T = x + y$：贡献为 0；
  - $T \in [l, r]$：贡献为 1；
  - 其他情况：贡献为 2。
- 可使用差分，快速维护区间加法操作。
  - 差分还原后，答案即为各 T 值的操作次数最小值：

```
1 d[2] += 2;  d[2 * m + 1] -= 2;
2 --d[l];     ++d[r + 1];
3 --d[x + y]; ++d[x + y + 1];
```

- 时间复杂度 $\mathcal{O}(n + m)$，期望得分 100 分。

题目大意

- 给定 n 个点的无向图，电压 a_i。
- $|a_i - a_j| \le d$ 时，连边 $i \leftrightarrow j$。
- q 次询问，求 u, v 的最短路，或判断不连通。

得分情况

- 参考难度：提高。

问题分析

- 注意到测试点 $1 \sim 7$：$n, q \leq 100$。

问题分析

- 注意到测试点 $1 \sim 7$：$n, q \leq 100$。
  - 点数较少时，可建图后预处理，或每次求最短路。

问题分析

- 注意到测试点 $1 \sim 7$：$n, q \leq 100$。
  - 点数较少时，可建图后预处理，或每次求最短路。
- 时间复杂度 $\mathcal{O}(n^3)/\mathcal{O}(n^2q)$，期望得分 35 分。

问题分析

- 注意到测试点 $1 \sim 7$：$n, q \leq 100$。
  - 点数较少时，可建图后预处理，或每次求最短路。
- 时间复杂度 $\mathcal{O}(n^3)/\mathcal{O}(n^2q)$，期望得分 35 分。
- 能否连边只由 a_i 之差决定，可将点按 a_i 升序排序：
  - 此时，各点的邻接点（包括点自身）形成一段连续区间；

问题分析

- 注意到测试点 $1 \sim 7$：$n, q \leq 100$。
  - 点数较少时，可建图后预处理，或每次求最短路。
- 时间复杂度 $\mathcal{O}(n^3)/\mathcal{O}(n^2q)$，期望得分 35 分。
- 能否连边只由 a_i 之差决定，可将点按 a_i 升序排序：
  - 此时，各点的邻接点（包括点自身）形成一段连续区间；
  - 每个连通块也是一段连续区间。

问题分析

- 注意到测试点 $1 \sim 7$：$n, q \leq 100$。
  - 点数较少时，可建图后预处理，或每次求最短路。
- 时间复杂度 $\mathcal{O}(n^3)/\mathcal{O}(n^2q)$，期望得分 35 分。
- 能否连边只由 a_i 之差决定，可将点按 a_i 升序排序：
  - 此时，各点的邻接点（包括点自身）形成一段连续区间；
  - 每个连通块也是一段连续区间。
- 最坏情况下，边数高达 $\mathcal{O}(n^2)$。
  - 若局限于图论，则难以突破边数的瓶颈；
  - 考虑挖掘图的特殊性质。

问题分析

- 注意到测试点 $8 \sim 12$：$n, q \leq 10^4$。

问题分析

- 注意到测试点 $8 \sim 12$：$n, q \leq 10^4$。
  - 考虑 $\mathcal{O}(nq)$ 的做法。

问题分析

- 注意到测试点 $8 \sim 12$：$n, q \leq 10^4$。
  - 考虑 $\mathcal{O}(nq)$ 的做法。
- 能否连边只由 a_i 之差决定，可将点按 a_i 升序排序：
  - 此时，各点的邻接点（包括点自身）形成一段连续区间；

问题分析

- 注意到测试点 $8 \sim 12$：$n, q \leq 10^4$。
  - 考虑 $\mathcal{O}(nq)$ 的做法。
- 能否连边只由 a_i 之差决定，可将点按 a_i 升序排序：
  - 此时，各点的邻接点（包括点自身）形成一段连续区间；
  - 每个连通块也是一段连续区间。

问题分析

- 注意到测试点 $8 \sim 12$：$n, q \leq 10^4$。
  - 考虑 $\mathcal{O}(nq)$ 的做法。
- 能否连边只由 a_i 之差决定，可将点按 a_i 升序排序：
  - 此时，各点的邻接点（包括点自身）形成一段连续区间；
  - 每个连通块也是一段连续区间。
- 查询时，若 u, v 不在同一个连通块，则无解。

问题分析

- 注意到测试点 $8 \sim 12$：$n, q \leq 10^4$。
  - 考虑 $\mathcal{O}(nq)$ 的做法。
- 能否连边只由 a_i 之差决定，可将点按 a_i 升序排序：
  - 此时，各点的邻接点（包括点自身）形成一段连续区间；
  - 每个连通块也是一段连续区间。
- 查询时，若 u, v 不在同一个连通块，则无解。
- 贪心：若块中 u 在 v 左侧，则 u 每次向后跳尽可能远，跳至 v 右侧的次数即为答案。

问题分析

- 为实现贪心，需要知道：
  - 点 u 在排序后的块中下标 $rk[u]$；

问题分析

- 为实现贪心，需要知道：
  - 点 u 在排序后的块中下标 $rk[u]$；
  - 块中下标位置 i 一步可跳到的最右位置 $r[i]$；

问题分析

- 为实现贪心，需要知道：
  - 点 u 在排序后的块中下标 $rk[u]$；
  - 块中下标位置 i 一步可跳到的最右位置 $r[i]$；
- 排序后，可使用双指针，$\mathcal{O}(n)$ 求出 $r[i]$。

问题分析

- 为实现贪心，需要知道：
  - 点 u 在排序后的块中下标 $rk[u]$；
  - 块中下标位置 i 一步可跳到的最右位置 $r[i]$；
- 排序后，可使用双指针，$\mathcal{O}(n)$ 求出 $r[i]$。
- 查询时，记 $x = rk[u]$，$y = rk[v]$。
- x, y 不在同一块时无解，否则贪心右跳，求出步数：

```
1 int cur = x, ans = 0;
2 while (cur < y) cur = r[cur], ++ans;
```

问题分析

- 为实现贪心，需要知道：
  - 点 u 在排序后的块中下标 $rk[u]$；
  - 块中下标位置 i 一步可跳到的最右位置 $r[i]$；
- 排序后，可使用双指针，$\mathcal{O}(n)$ 求出 $r[i]$。
- 查询时，记 $x = rk[u]$，$y = rk[v]$。
- x, y 不在同一块时无解，否则贪心右跳，求出步数：

```
1 int cur = x, ans = 0;
2 while (cur < y) cur = r[cur], ++ans;
```

- 时间复杂度 $\mathcal{O}(n \log n + nq)$，期望得分 60 分。

问题分析

- 注意到特殊性质 A：$d = 0$。

问题分析

- 注意到特殊性质 A：$d = 0$。
  - 电压相同时才连边，每个连通块都是完全图；

问题分析

- 注意到特殊性质 A：$d = 0$。
  - 电压相同时才连边，每个连通块都是完全图；
  - $u = v$ 时，但为 0，$a_u = a_v$ 为 1，否则 -1。

问题分析

- 注意到特殊性质 A：$d = 0$。
  - 电压相同时才连边，每个连通块都是完全图；
  - $u = v$ 时，但为 0，$a_u = a_v$ 为 1，否则 -1。
- 注意到特殊性质 B：电压非降序，且相邻差 $\leq d$、隔一个差 $> d$。

问题分析

- 注意到特殊性质 A：$d = 0$。
  - 电压相同时才连边，每个连通块都是完全图；
  - $u = v$ 时，但为 0，$a_u = a_v$ 为 1，否则 -1。
- 注意到特殊性质 B：电压非降序，且相邻差 $\leq d$、隔一个差 $> d$。
  - 图是一条链，相邻两点直接连边，隔一个及以上不连；

问题分析

- 注意到特殊性质 A：$d = 0$。
  - 电压相同时才连边，每个连通块都是完全图；
  - $u = v$ 时，但为 0，$a_u = a_v$ 为 1，否则 -1。
- 注意到特殊性质 B：电压非降序，且相邻差 $\leq d$、隔一个差 $> d$。
  - 图是一条链，相邻两点直接连边，隔一个及以上不连；
  - 答案：$u = v$ 为 0，否则为 $|u - v|$。

问题分析

- 注意到特殊性质 A：$d = 0$。
  - 电压相同时才连边，每个连通块都是完全图；
  - $u = v$ 时，但为 0，$a_u = a_v$ 为 1，否则 -1。
- 注意到特殊性质 B：电压非降序，且相邻差 $\leq d$、隔一个差 $> d$。
  - 图是一条链，相邻两点直接连边，隔一个及以上不连；
  - 答案：$u = v$ 为 0，否则为 $|u - v|$。
- 瓶颈：段内逐次后跳，单次询问最坏 $\mathcal{O}(n)$。

问题分析

- 注意到特殊性质 A：$d = 0$。
  - 电压相同时才连边，每个连通块都是完全图；
  - $u = v$ 时，但为 0，$a_u = a_v$ 为 1，否则 -1。
- 注意到特殊性质 B：电压非降序，且相邻差 $\leq d$、隔一个差 $> d$。
  - 图是一条链，相邻两点直接连边，隔一个及以上不连；
  - 答案：$u = v$ 为 0，否则为 $|u - v|$。
- 瓶颈：段内逐次后跳，单次询问最坏 $\mathcal{O}(n)$。
- 可使用倍增，加速形如 $r\left[r[i]\right]$ 的后跳过程。

问题分析

- $jump[k][i]$：块中位置 i 向右跳 2^k 次的最右位置。
  - 初始状态：$jump[0][i] = r[i]$;

问题分析

- $jump[k][i]$：块中位置 i 向右跳 2^k 次的最右位置。
  - 初始状态：$jump[0][i] = r[i]$；
  - 状态转移方程：

$$jump[k][i] = jump[k-1]\Big[jump[k-1][i]\Big]$$

问题分析

- $jump[k][i]$：块中位置 i 向右跳 2^k 次的最右位置。
  - 初始状态：$jump[0][i] = r[i]$；
  - 状态转移方程：

$$jump[k][i] = jump[k-1]\Big[jump[k-1][i]\Big]$$

- 查询时，由 u, v 定位到块中 x, y，从 x 倍增贪心跳，尽量不越过 y，统计步数：

```
1  int cur = x, ans = 0;
2  for (int k = LOG - 1; k >= 0; --k)
3    if (jump[k][cur] < y)
4      cur = jump[k][cur], ans += 1 << k;
5  if (jump[0][cur] >= y) cout << ans + 1 << '\n';
```

问题分析

- $jump[k][i]$：块中位置 i 向右跳 2^k 次的最右位置。
  - 初始状态：$jump[0][i] = r[i]$；
  - 状态转移方程：

$$jump[k][i] = jump[k-1]\Big[jump[k-1][i]\Big]$$

- 查询时，由 u, v 定位到块中 x, y，从 x 倍增贪心跳，尽量不越过 y，统计步数：

```
1  int cur = x, ans = 0;
2  for (int k = LOG - 1; k >= 0; --k)
3    if (jump[k][cur] < y)
4      cur = jump[k][cur], ans += 1 << k;
5  if (jump[0][cur] >= y) cout << ans + 1 << '\n';
```

- 时间复杂度 $\mathcal{O}((n+q)\log n)$，期望得分 100 分。

题目大意

- 给定长为 n 的 01 规则串 s 与长为 m 的模板串 t。
- 构造长度为 $n+m-1$ 的小写字符串 w，对每个 $1 \le i \le n$:
  - $s_i = 1$: $w_i \cdots w_{i+m-1} = t$;
  - $s_i = 0$: $w_i \cdots w_{i+m-1} \neq t$。
- 求字典序最小的 w，无解输出 -1。

得分情况

- 参考难度：提高 +/省选-。

得分情况

- 参考难度：提高 +/省选-。
- 测试点 $1 \sim 4$：$n, m \leq 10$。
- 测试点 $5 \sim 8$：$n, m \leq 5000$。
- 测试点 $9 \sim 12$：所有 $s_i = 0$。
- 测试点 $13 \sim 16$：所有 $t_j = \mathtt{a}$。
- 测试点 $17 \sim 25$：完整数据，$n, m \leq 10^6$。

问题分析

- 注意到测试点 $1 \sim 4$：$n, m \leq 10$。

问题分析

- 注意到测试点 $1 \sim 4$：$n, m \leq 10$。
  - 从左到右枚举 w 的每一位，共 26^{n+m-1} 种可能；
  - 枚举完成后，逐个检查 s_i 对应的长度为 m 的子串；
  - 按字典序从小到大枚举，第一次找到的就是答案。

问题分析

- 注意到测试点 $1 \sim 4$：$n, m \leq 10$。
  - 从左到右枚举 w 的每一位，共 26^{n+m-1} 种可能；
  - 枚举完成后，逐个检查 s_i 对应的长度为 m 的子串；
  - 按字典序从小到大枚举，第一次找到的就是答案。
- 时间复杂度 $\mathcal{O}(26^{n+m-1} \cdot nm)$，期望得分 16 分。

问题分析

- 注意到测试点 $1 \sim 8$：$n, m \leq 5000$。

问题分析

- 注意到测试点 $1 \sim 8$：$n, m \leq 5000$。
  - 不能枚举整个字符串，考虑直接处理所有必须等于 t 的窗口。

问题分析

- 注意到测试点 $1 \sim 8$：$n, m \leq 5000$。
  - 不能枚举整个字符串，考虑直接处理所有必须等于 t 的窗口。
- 先处理 $s_i = 1$ 的位置：
  - 把 t 填入 $w_i \cdots w_{i+m-1}$；
  - 同一位置被多次填入时，若字符不同则无解；
  - 没有被确定的位置先填成字典序最小的字符 a。

问题分析

- 注意到测试点 $1 \sim 8$：$n, m \leq 5000$。
  - 不能枚举整个字符串，考虑直接处理所有必须等于 t 的窗口。
- 先处理 $s_i = 1$ 的位置：
  - 把 t 填入 $w_i \cdots w_{i+m-1}$；
  - 同一位置被多次填入时，若字符不同则无解；
  - 没有被确定的位置先填成字典序最小的字符 a。
- 这样得到一个已经满足所有 1 约束的字符串。

问题分析

- 再从左到右处理 $s_i = 0$ 的窗口。
  - 若当前窗口已经不等于 t，不需要修改；
  - 若当前窗口恰好等于 t，把窗口内最靠后的待定位置改为 b；
  - 若窗口中没有待定位置，则与某个 $s_j = 1$ 的窗口矛盾，无解。

问题分析

- 再从左到右处理 $s_i = 0$ 的窗口。
  - 若当前窗口已经不等于 t，不需要修改；
  - 若当前窗口恰好等于 t，把窗口内最靠后的待定位置改为 b；
  - 若窗口中没有待定位置，则与某个 $s_j = 1$ 的窗口矛盾，无解。
- 为什么要改最靠后的位置？
  - 当前窗口的前面部分已经确定，越靠后的位置越不会影响答案的更小字典序前缀；
  - 只把 a 改成 b，是满足当前约束所需的最小字典序改变。

问题分析

- 再从左到右处理 $s_i = 0$ 的窗口。
  - 若当前窗口已经不等于 t，不需要修改；
  - 若当前窗口恰好等于 t，把窗口内最靠后的待定位置改为 b；
  - 若窗口中没有待定位置，则与某个 $s_j = 1$ 的窗口矛盾，无解。
- 为什么要改最靠后的位置？
  - 当前窗口的前面部分已经确定，越靠后的位置越不会影响答案的更小字典序前缀；
  - 只把 a 改成 b，是满足当前约束所需的最小字典序改变。
- 朴素地判断每个窗口并寻找待定位置，时间复杂度 $\mathcal{O}(nm)$，累计得分 32 分。

问题分析

- 上述贪心需要说明两件事。

问题分析

- 上述贪心需要说明两件事。
- 问题一：修改当前窗口，是否会使之前处理过的 0 窗口重新变成 t?
  - 若之前的窗口与当前窗口重叠，当前窗口中被修改的位置位于之前窗口的右侧；
  - 若它在之前窗口中，之前窗口原本已经被改成不等于 t，修改只会让该位置变成 b；
  - 因此之前的 0 窗口不会重新变成 t。

问题分析

- 上述贪心需要说明两件事。
- 问题一：修改当前窗口，是否会使之前处理过的 0 窗口重新变成 t?
  - 若之前的窗口与当前窗口重叠，当前窗口中被修改的位置位于之前窗口的右侧；
  - 若它在之前窗口中，之前窗口原本已经被改成不等于 t，修改只会让该位置变成 b；
  - 因此之前的 0 窗口不会重新变成 t。
- 问题二：为什么只需要改成 b?
  - 当前窗口本来等于 t，而 t 中对应位置只能是 a；
  - 改成 b 后当前窗口立即不等于 t，并且字典序上是最小的可行修改；
  - 改成 c 或更大的字符不会带来额外收益。

问题分析

- 注意到测试点 $9 \sim 12$：所有 $s_i = 0$。

问题分析

- 注意到测试点 $9 \sim 12$：所有 $s_i = 0$。
  - 若 t 中存在非 a 字符，令 w 的每一位都是 a，则所有窗口都不等于 t；
  - 若 $t = \mathtt{a}^m$，需要让每个长度为 m 的窗口至少包含一个 b。

问题分析

- 注意到测试点 $9 \sim 12$：所有 $s_i = 0$。
  - 若 t 中存在非 a 字符，令 w 的每一位都是 a，则所有窗口都不等于 t；
  - 若 $t = \mathtt{a}^m$，需要让每个长度为 m 的窗口至少包含一个 b。
- 对于第二种情况，把
  - $w_m, w_{2m}, w_{3m}, \ldots$ 改为 b；
  - 其余位置设为 a。

问题分析

- 注意到测试点 $9 \sim 12$：所有 $s_i = 0$。
  - 若 t 中存在非 a 字符，令 w 的每一位都是 a，则所有窗口都不等于 t；
  - 若 $t = \mathtt{a}^m$，需要让每个长度为 m 的窗口至少包含一个 b。
- 对于第二种情况，把
  - $w_m, w_{2m}, w_{3m}, \ldots$ 改为 b；
  - 其余位置设为 a。
- 每个长度为 m 的窗口都包含一个上述位置，且这些 b 尽可能靠后，故字典序最小。
- 时间复杂度 $\mathcal{O}(n+m)$，累计得分 48 分。

问题分析

- 注意到测试点 $13 \sim 16$：所有 $t_j = \mathtt{a}$。

问题分析

- 注意到测试点 $13 \sim 16$：所有 $t_j = \mathtt{a}$。
  - $s_i = 1$ 的窗口要求其中所有位置都被锁定为 a；
  - $s_i = 0$ 的窗口要求至少存在一个 b。

问题分析

- 注意到测试点 $13 \sim 16$：所有 $t_j = \mathtt{a}$。
  - $s_i = 1$ 的窗口要求其中所有位置都被锁定为 a；
  - $s_i = 0$ 的窗口要求至少存在一个 b。
- 用差分数组标记所有 $s_i = 1$ 窗口覆盖的位置，得到每个位置是否被锁定。

问题分析

- 注意到测试点 $13 \sim 16$：所有 $t_j = \mathtt{a}$。
  - $s_i = 1$ 的窗口要求其中所有位置都被锁定为 a；
  - $s_i = 0$ 的窗口要求至少存在一个 b。
- 用差分数组标记所有 $s_i = 1$ 窗口覆盖的位置，得到每个位置是否被锁定。
- 从左到右处理 $s_i = 0$：
  - 窗口内已有 b 时直接跳过；
  - 否则把窗口内最靠后的未锁定位置改成 b；
  - 可以用数组 $preQ$ 维护每个位置左侧最近的未锁定位置。

问题分析

- 注意到测试点 $13 \sim 16$：所有 $t_j = \mathtt{a}$。
  - $s_i = 1$ 的窗口要求其中所有位置都被锁定为 a；
  - $s_i = 0$ 的窗口要求至少存在一个 b。
- 用差分数组标记所有 $s_i = 1$ 窗口覆盖的位置，得到每个位置是否被锁定。
- 从左到右处理 $s_i = 0$：
  - 窗口内已有 b 时直接跳过；
  - 否则把窗口内最靠后的未锁定位置改成 b；
  - 可以用数组 $preQ$ 维护每个位置左侧最近的未锁定位置。
- 时间复杂度 $\mathcal{O}(n+m)$，累计得分 64 分。

问题分析

- 注意到测试点 $17 \sim 25$：完整数据，$n, m \leq 10^6$。

问题分析

- 注意到测试点 $17 \sim 25$：完整数据，$n, m \leq 10^6$。
  - 需要快速判断两个 t 的重叠是否冲突；
  - 需要快速判断当前答案的某个窗口是否等于 t；
  - 需要快速找到窗口内最靠后的待定位置。

问题分析

- 注意到测试点 $17 \sim 25$：完整数据，$n, m \leq 10^6$。
  - 需要快速判断两个 t 的重叠是否冲突；
  - 需要快速判断当前答案的某个窗口是否等于 t；
  - 需要快速找到窗口内最靠后的待定位置。
- 对两个相距 d 的 $s_i = s_j = 1$ 窗口：
  - 若 $d \geq m$，两个窗口没有重叠；
  - 若 $d < m$，需要判断 t 的后缀 $t_{d+1} \cdots t_m$ 是否等于前缀 $t_1 \cdots t_{m-d}$。

问题分析

- 注意到测试点 $17 \sim 25$：完整数据，$n, m \leq 10^6$。
  - 需要快速判断两个 t 的重叠是否冲突；
  - 需要快速判断当前答案的某个窗口是否等于 t；
  - 需要快速找到窗口内最靠后的待定位置。
- 对两个相距 d 的 $s_i = s_j = 1$ 窗口：
  - 若 $d \geq m$，两个窗口没有重叠；
  - 若 $d < m$，需要判断 t 的后缀 $t_{d+1} \cdots t_m$ 是否等于前缀 $t_1 \cdots t_{m-d}$。
- 预处理 t 的 Z 函数，便可在 $\mathcal{O}(1)$ 时间判断重叠是否冲突。

问题分析

- 初步处理所有 $s_i = 1$ 后，把未确定位置设为 a。

问题分析

- 初步处理所有 $s_i = 1$ 后，把未确定位置设为 a。
- 令 $q = t + \# + w$，计算 q 的 Z 函数：
  - $w_i \cdots w_{i+m-1} = t$
  - $\iff z_{m+1+i} \geq m$。

问题分析

- 初步处理所有 $s_i = 1$ 后，把未确定位置设为 a。
- 令 $q = t + \# + w$，计算 q 的 Z 函数：
  - $w_i \cdots w_{i+m-1} = t$
  - $\iff z_{m+1+i} \geq m$。
- 为每个位置维护并查集：
  - 集合中保存还没有被锁定、也没有被改成 b 的位置；
  - 查询 $[i, i+m-1]$ 中最靠后的可用位置，并将其删除；
  - 已经处理过的 0 窗口不需要重复修改。

问题分析

- 初步处理所有 $s_i = 1$ 后，把未确定位置设为 a。
- 令 $q = t + \# + w$，计算 q 的 Z 函数：
  - $w_i \cdots w_{i+m-1} = t$
  - $\iff z_{m+1+i} \geq m$。
- 为每个位置维护并查集：
  - 集合中保存还没有被锁定、也没有被改成 b 的位置；
  - 查询 $[i, i+m-1]$ 中最靠后的可用位置，并将其删除；
  - 已经处理过的 0 窗口不需要重复修改。
- 找不到可用位置时输出 -1，否则得到字典序最小的答案。

问题分析

- Z 函数负责匹配，差分或并查集负责维护可修改位置：

```
1  getz(t, m, z);                          // 判断 1 窗口的重叠
2  getz(t + '#' + w, n + 2 * m, z);        // 判断 0 窗口是否等于 t
3  p = find(i + m);                        // 找窗口内最靠后的可用
4  if (p == 0 || p - 1 < i) cout << "-1";
5  else w[p - 1] = 'b';
```

问题分析

- Z 函数负责匹配，差分或并查集负责维护可修改位置：

```
1  getz(t, m, z);                              // 判断 1 窗口的重叠
2  getz(t + '#' + w, n + 2 * m, z);  // 判断 0 窗口是否等于 t
3  p = find(i + m);                            // 找窗口内最靠后的可用
4  if (p == 0 || p - 1 < i) cout << "-1";
5  else w[p - 1] = 'b';
```

- 每个位置最多被处理一次，Z 函数、并查集和贪心扫描均为线性复杂度。

问题分析

- Z 函数负责匹配，差分或并查集负责维护可修改位置：

```
1  getz(t, m, z);                              // 判断 1 窗口的重叠
2  getz(t + '#' + w, n + 2 * m, z);  // 判断 0 窗口是否等于 t
3  p = find(i + m);                            // 找窗口内最靠后的可用
4  if (p == 0 || p - 1 < i) cout << "-1";
5  else w[p - 1] = 'b';
```

- 每个位置最多被处理一次，Z 函数、并查集和贪心扫描均为线性复杂度。
- 时间复杂度 $\mathcal{O}(n+m)$，空间复杂度 $\mathcal{O}(n+m)$，期望得分 100 分。

题目大意

- 给定长度为 n 的序列 $a_1, \ldots, a_n$，共有 $\frac{n(n+1)}{2}$ 个连续子段。
- 子段极差定义为 $\max - \min$。
- 选出极差最大的 k 个不同子段，求这 k 个极差之和的最大值。

得分情况

- 参考难度：省选/NOI-。

得分情况

- 参考难度：省选/NOI-。
- 测试点 1：$n \le 10, k \le 5$。
- 测试点 $2 \sim 3$：$n \le 100, k \le 100$。
- 测试点 $4 \sim 9$：$n \le 10^5, k \le 10^5$。
- 测试点 $10 \sim 13$：$a_1 \le a_2 \le \cdots \le a_n$。
- 测试点 $14 \sim 16$：$k = \dfrac{n(n+1)}{2}$。
- 测试点 $17 \sim 20$：$k \le 10^9$。
- 测试点 $21 \sim 25$：$k \le 10^{12}$。

问题分析

- 注意到测试点 $1 \sim 3$：$n \leq 100, k \leq 100$。

问题分析

- 注意到测试点 $1 \sim 3$：$n \leq 100, k \leq 100$。
  - 枚举所有连续子段；
  - 向右延伸时维护当前子段的最大值和最小值；
  - 将所有极差排序，取前 k 大求和。

问题分析

- 注意到测试点 $1 \sim 3$：$n \leq 100, k \leq 100$。
  - 枚举所有连续子段；
  - 向右延伸时维护当前子段的最大值和最小值；
  - 将所有极差排序，取前 k 大求和。
- 时间复杂度 $\mathcal{O}(n^2 \log n)$，空间复杂度 $\mathcal{O}(n^2)$，期望得分 12 分。

问题分析

- 注意到测试点 $4 \sim 9$：$n \leq 10^5, k \leq 10^5$。

问题分析

- 注意到测试点 $4 \sim 9$：$n \leq 10^5, k \leq 10^5$。
  - 不能保存所有子段，考虑如何快速取出最大的 k 个极差。

问题分析

- 注意到测试点 $4 \sim 9$：$n \leq 10^5, k \leq 10^5$。
  - 不能保存所有子段，考虑如何快速取出最大的 k 个极差。
- 固定左端点 l，令右端点从 n 向左移动：
  - 子段逐渐缩短，极差单调不增；
  - 因此每个 l 对应一个有序序列。

问题分析

- 注意到测试点 $4 \sim 9$：$n \leq 10^5, k \leq 10^5$。
  - 不能保存所有子段，考虑如何快速取出最大的 k 个极差。
- 固定左端点 l，令右端点从 n 向左移动：
  - 子段逐渐缩短，极差单调不增；
  - 因此每个 l 对应一个有序序列。
- ST 表可以在 $\mathcal{O}(1)$ 时间求出任意区间的最大值、最小值。

问题分析

- 对每个左端点 l，先把区间 $[l, n]$ 放入最大堆。

问题分析

- 对每个左端点 l，先把区间 $[l, n]$ 放入最大堆。
- 每次取出堆顶 (l, r)：
  - 当前子段 $[l, r]$ 是所有未取子段中极差最大的；
  - 将同一条序列的下一个元素 $(l, r-1)$ 加入最大堆。

问题分析

- 对每个左端点 l，先把区间 $[l, n]$ 放入最大堆。
- 每次取出堆顶 (l, r)：
  - 当前子段 $[l, r]$ 是所有未取子段中极差最大的；
  - 将同一条序列的下一个元素 $(l, r-1)$ 加入最大堆。
- 重复 k 次即可得到答案。

```
1 for (int l = 1; l <= n; ++l) q.push({val(l, n), l, n});
2 for (int i = 1; i <= k; ++i) {
3   x = q.top(); q.pop(); ans += x.v;
4   if (x.l < x.r) q.push({val(x.l, x.r - 1), x.l, x.r - 1})
5 }
```

问题分析

- 对每个左端点 l，先把区间 $[l,n]$ 放入最大堆。
- 每次取出堆顶 (l,r)：
  - 当前子段 $[l,r]$ 是所有未取子段中极差最大的；
  - 将同一条序列的下一个元素 $(l,r-1)$ 加入最大堆。
- 重复 k 次即可得到答案。

```
1 for (int l = 1; l <= n; ++l) q.push({val(l, n), l, n});
2 for (int i = 1; i <= k; ++i) {
3   x = q.top(); q.pop(); ans += x.v;
4   if (x.l < x.r) q.push({val(x.l, x.r - 1), x.l, x.r - 1})
5 }
```

- 时间复杂度 $\mathcal{O}((n+k)\log n)$，累计得分 36 分。

问题分析

- 注意到测试点 $10 \sim 13$：a 非降序。

问题分析

- 注意到测试点 $10 \sim 13$：a 非降序。
  - 子段 $[l, r]$ 的极差为 $a_r - a_l$；
  - 问题变成选 k 个下标对 (l, r)，最大化 $\sum(a_r - a_l)$。

问题分析

- 注意到测试点 $10 \sim 13$：a 非降序。
  - 子段 $[l, r]$ 的极差为 $a_r - a_l$；
  - 问题变成选 k 个下标对 (l, r)，最大化 $\sum(a_r - a_l)$。
- 二分第 k 大的差值 V。
  - 双指针求 $a_r - a_l \geq x$ 的下标对数量；
  - 前缀和求这些下标对的差值之和。

问题分析

- 注意到测试点 $10 \sim 13$：a 非降序。
  - 子段 $[l, r]$ 的极差为 $a_r - a_l$；
  - 问题变成选 k 个下标对 (l, r)，最大化 $\sum(a_r - a_l)$。
- 二分第 k 大的差值 V。
  - 双指针求 $a_r - a_l \geq x$ 的下标对数量；
  - 前缀和求这些下标对的差值之和。
- 时间复杂度 $\mathcal{O}(n \log V)$，累计得分 64 分。

问题分析

- 注意到测试点 $14 \sim 16$：$k = \dfrac{n(n+1)}{2}$，需要选出全部子段。

问题分析

- 注意到测试点 $14 \sim 16$：$k = \dfrac{n(n+1)}{2}$，需要选出全部子段。
- 考虑每个 a_i 对答案的贡献：
  - 作为最大值时，贡献为 $+a_i$；
  - 作为最小值时，贡献为 $-a_i$。

问题分析

- 注意到测试点 $14 \sim 16$：$k = \dfrac{n(n+1)}{2}$，需要选出全部子段。
- 考虑每个 a_i 对答案的贡献：
  - 作为最大值时，贡献为 $+a_i$；
  - 作为最小值时，贡献为 $-a_i$。
- 用两组单调栈求出 a_i 作为最大值、最小值时的子段数量。

```
1  ans += a[i] * (1LL * (i - lmx[i]) * (rmx[i] - i)
2                 - 1LL * (i - lmn[i]) * (rmn[i] - i));
```

问题分析

- 注意到测试点 $14 \sim 16$：$k = \dfrac{n(n+1)}{2}$，需要选出全部子段。
- 考虑每个 a_i 对答案的贡献：
  - 作为最大值时，贡献为 $+a_i$；
  - 作为最小值时，贡献为 $-a_i$。
- 用两组单调栈求出 a_i 作为最大值、最小值时的子段数量。

```
1  ans += a[i] * (1LL * (i - lmx[i]) * (rmx[i] - i)
2                - 1LL * (i - lmn[i]) * (rmn[i] - i));
```

- 时间复杂度 $\mathcal{O}(n)$，累计得分 64 分。

问题分析

- 注意到测试点 $17 \sim 25$：无特殊性质，k 可达到 10^{12}。

问题分析

- 注意到测试点 $17 \sim 25$：无特殊性质，k 可达到 10^{12}。
  - 不能枚举所有子段，也不能把前 k 个极差全部放进堆；
  - 考虑二分第 k 大的极差 V。

问题分析

- 注意到测试点 $17 \sim 25$：无特殊性质，k 可达到 10^{12}。
  - 不能枚举所有子段，也不能把前 k 个极差全部放进堆；
  - 考虑二分第 k 大的极差 V。
- 设 $\texttt{cnt}(x)$ 为极差至少为 x 的子段数量。
  - 若 $\texttt{cnt}(x) \geq k$，第 k 大极差至少为 x；
  - 否则第 k 大极差小于 x。

问题分析

- 注意到测试点 $17 \sim 25$：无特殊性质，k 可达到 10^{12}。
  - 不能枚举所有子段，也不能把前 k 个极差全部放进堆；
  - 考虑二分第 k 大的极差 V。
- 设 $\texttt{cnt}(x)$ 为极差至少为 x 的子段数量。
  - 若 $\texttt{cnt}(x) \geq k$，第 k 大极差至少为 x；
  - 否则第 k 大极差小于 x。
- 用单调队列维护当前窗口最大值和最小值，双指针求 $\texttt{cnt}(x)$，单次检查为 $\mathcal{O}(n)$。

问题分析

- 二分得到第 k 大极差 V 后，先处理所有极差严格大于 V 的子段。

问题分析

- 二分得到第 k 大极差 V 后，先处理所有极差严格大于 V 的子段。
  - 令 $c = \mathrm{cnt}(V+1)$;
  - 这些子段一定全部选入，设它们的极差和为 S;
  - 还需要从极差等于 V 的子段中选 $k-c$ 个。

问题分析

- 二分得到第 k 大极差 V 后，先处理所有极差严格大于 V 的子段。
  - 令 $c = \mathrm{cnt}(V+1)$;
  - 这些子段一定全部选入，设它们的极差和为 S;
  - 还需要从极差等于 V 的子段中选 $k-c$ 个。
- 因而最终答案为

$$\mathrm{ans} = S + V \times (k - \mathrm{cnt}(V+1))$$

问题分析

- 二分得到第 k 大极差 V 后，先处理所有极差严格大于 V 的子段。
  - 令 $c = \mathrm{cnt}(V+1)$;
  - 这些子段一定全部选入，设它们的极差和为 S;
  - 还需要从极差等于 V 的子段中选 $k-c$ 个。
- 因而最终答案为

$$\mathrm{ans} = S + V \times (k - \mathrm{cnt}(V+1))$$

- 关键变为计算 $S = \mathrm{sum}(V+1)$。

问题分析

- 固定右端点 r，从左到右移动左端点 l：
  - 区间最大值只会变小，区间最小值只会变大；
  - 因而最大值、最小值分别在若干段连续的 l 上保持不变。

问题分析

- 固定右端点 r，从左到右移动左端点 l：
  - 区间最大值只会变小，区间最小值只会变大；
  - 因而最大值、最小值分别在若干段连续的 l 上保持不变。
- 用单调栈维护这些分段：
  - 栈中保存每一段的值和左端点数量；
  - 新元素加入时，弹出被它覆盖的若干段并合并数量；
  - 栈上前缀和保存各段的数量和数值贡献。

问题分析

- 固定右端点 r，从左到右移动左端点 l：
  - 区间最大值只会变小，区间最小值只会变大；
  - 因而最大值、最小值分别在若干段连续的 l 上保持不变。
- 用单调栈维护这些分段：
  - 栈中保存每一段的值和左端点数量；
  - 新元素加入时，弹出被它覆盖的若干段并合并数量；
  - 栈上前缀和保存各段的数量和数值贡献。
- 用线段树二分找到满足极差至少为 x 的最左端点，再查询前若干段的贡献和。

问题分析

- 对固定的 r，用单调栈得到：
  - 所有左端点对应的最大值分段；
  - 所有左端点对应的最小值分段。

问题分析

- 对固定的 r，用单调栈得到：
  - 所有左端点对应的最大值分段；
  - 所有左端点对应的最小值分段。
- 对每一段，区间极差等于「最大值分段的值」减去「最小值分段的值」。

问题分析

- 对固定的 r，用单调栈得到：
  - 所有左端点对应的最大值分段；
  - 所有左端点对应的最小值分段。
- 对每一段，区间极差等于「最大值分段的值」减去「最小值分段的值」。
  - 线段树维护每个左端点当前的极差；
  - 当最大值或最小值的一段整体变化时，对一个区间进行 Lazy 加法；
  - 维护区间和，并二分出极差达到阈值 x 的位置。

问题分析

- 对固定的 r，用单调栈得到：
  - 所有左端点对应的最大值分段；
  - 所有左端点对应的最小值分段。
- 对每一段，区间极差等于「最大值分段的值」减去「最小值分段的值」。
  - 线段树维护每个左端点当前的极差；
  - 当最大值或最小值的一段整体变化时，对一个区间进行 Lazy 加法；
  - 维护区间和，并二分出极差达到阈值 x 的位置。
- 这样每个右端点只需进行若干次单调栈合并和线段树操作。

问题分析

- 100 分做法的整体流程：
  1. 二分第 k 大的极差 V；
  2. 用单调队列求 $\mathrm{cnt}(V+1)$；
  3. 用单调栈维护最大值、最小值分段；
  4. 用 Lazy 线段树和线段树二分求 $\mathrm{sum}(V+1)$；
  5. 用极差为 V 的子段补齐答案。

问题分析

- 100 分做法的整体流程：
  1. 二分第 k 大的极差 V；
  2. 用单调队列求 $\mathtt{cnt}(V+1)$；
  3. 用单调栈维护最大值、最小值分段；
  4. 用 Lazy 线段树和线段树二分求 $\mathtt{sum}(V+1)$；
  5. 用极差为 V 的子段补齐答案。
- 二分部分复杂度为 $\mathcal{O}(n \log V)$；
- 每次计算分段贡献的复杂度为 $\mathcal{O}(n \log n)$；
- 总时间复杂度 $\mathcal{O}(n \log V + n \log n)$，空间复杂度 $\mathcal{O}(n)$，期望得分 100 分。

Thanks for your attention!